

实验四：词向量

word2vec 的梯度推导 · 中文词向量的训练、评测与可视化

姓名：李贵龙 学号：202411581243

2026 年 9 月 30 日

摘要

本次实验分为理论与编程两部分。理论部分补全 Lecture 3 第 35 页略去的推导：在朴素 softmax 下求损失对上下文向量 v_k 的偏导，理解梯度“期望减观测”的含义，并估算朴素 softmax 每一步的计算代价。编程部分在中文维基百科语料上完成“预处理与分词 → 训练 skip-gram + 负采样词向量 → 中文词相似度与最近邻评测 → 超参数对比 → PCA/t-SNE 可视化”的完整流程。训练使用官方 C 代码 [tmikolov/word2vec](#)；skip-gram 与负采样分别由 [Mikolov et al. \[2013a\]](#) 与 [Mikolov et al. \[2013b\]](#) 提出。

目录

零、先读这一节	2
1 第一部分理论 (40 分)	2
1.1 题 1(a)：对中心词向量求导 (10 分)	2
1.2 题 1(b)：对上下文向量求导 (18 分)	2
1.3 题 1(c)：理解梯度 (12 分)	3
2 第二部分编程 (55 分)	4
2.1 环境与数据	4
2.2 题 2(a)：语料预处理 (10 分)	4
2.3 题 2(b)：训练 (12 分)	6
2.4 题 2(c)：内在评测 (15 分)	7
2.5 题 2(d)：超参数对比 (10 分)	9
2.6 题 2(e)：可视化 (8 分)	10
3 第三部分提交信息	13

零、先读这一节

记号约定 (与讲义一致, 与 CS224n 相反): u_i 是词 i 作为中心词时的向量, v_i 是词 i 作为上下文词时的向量, $u_i, v_i \in \mathbb{R}^d$ 。对一个训练对 (中心词 t , 上下文词 c), 朴素 softmax 下的条件概率与损失为

$$P(k | t) = \frac{\exp(u_t \cdot v_k)}{\sum_{k' \in V} \exp(u_t \cdot v_{k'})}, \quad y = -\log P(c | t). \quad (1)$$

把所有上下文向量按行堆成矩阵 $V \in \mathbb{R}^{|V| \times d}$ (第 k 行是 $v_k^\top$); 记 $\hat{y} \in \mathbb{R}^{|V|}$ 为预测分布 (第 k 个分量是 $P(k | t)$), e_c 为第 c 个分量为 1 的 one-hot 向量。词向量的基础概念可参考 [Jurafsky and Martin \[2024\]](#) 第 6 章。

1 第一部分理论 (40 分)

1.1 题 1(a): 对中心词向量求导 (10 分)

先展开对数, 把损失写成两项之和。记配分函数 $Z = \sum_{k \in V} \exp(u_t \cdot v_k)$, 则

$$y = -\log \frac{\exp(u_t \cdot v_c)}{Z} = -(u_t \cdot v_c - \log Z) = -u_t \cdot v_c + \log \sum_{k \in V} \exp(u_t \cdot v_k). \quad (2)$$

对 u_t 逐项求导。第一项 $-u_t \cdot v_c = -\sum_{i=1}^d u_{t,i} v_{c,i}$, 由 $\partial(u_t \cdot v_c)/\partial u_t = v_c$, 得

$$\frac{\partial(-u_t \cdot v_c)}{\partial u_t} = -v_c. \quad (3)$$

第二项 $\log Z$ 用链式法则: 外层函数是 $\log(\cdot)$, 内层函数是 $Z(u_t) = \sum_k \exp(u_t \cdot v_k)$ 。因为

$$\frac{\partial Z}{\partial u_t} = \sum_{k \in V} \frac{\partial \exp(u_t \cdot v_k)}{\partial u_t} = \sum_{k \in V} \exp(u_t \cdot v_k) v_k, \quad (4)$$

所以

$$\frac{\partial \log Z}{\partial u_t} = \frac{1}{Z} \frac{\partial Z}{\partial u_t} = \sum_{k \in V} \frac{\exp(u_t \cdot v_k)}{Z} v_k = \sum_{k \in V} P(k | t) v_k. \quad (5)$$

合起来即得

$$\boxed{\frac{\partial y}{\partial u_t} = -v_c + \sum_{k \in V} P(k | t) v_k} \quad (6)$$

矩阵形式。由 V 的行是 $v_k^\top$ 可知 $\sum_k P(k | t) v_k = V^\top \hat{y}$, 而 $v_c = V^\top e_c$, 因此

$$\frac{\partial y}{\partial u_t} = V^\top (\hat{y} - e_c) \in \mathbb{R}^d. \quad (7)$$

1.2 题 1(b): 对上下文向量求导 (18 分)

由式 (2), y 对 v_k 的依赖来自两部分: 第一项 $-u_t \cdot v_c$ 与对数配分项 $\log Z$ 。

(i) 第一项 $-u_t \cdot v_c$ 的贡献。 $-u_t \cdot v_c$ 中只出现 v_c , 与其它 v_k ($k \neq c$) 无关, 因此

$$\frac{\partial(-u_t \cdot v_c)}{\partial v_k} = \begin{cases} -u_t, & k = c, \\ 0, & k \neq c. \end{cases} \quad (8)$$

这正是“第一项 $-u_t \cdot v_c$ 只在 $k = c$ 时有贡献”的原因。

(ii) 对 $\log Z$ 使用链式法则。把每个 v_k 都看作自变量：

- 外层函数： $f(g) = \log g$ ，其导数为 $f'(g) = 1/g$ ，其中 $g = Z$ ；
- 内层函数： $g(v_k) = Z = \sum_{j \in V} \exp(u_t \cdot v_j)$ ，只有 $j = k$ 这一项与 v_k 有关，故

$$\frac{\partial g}{\partial v_k} = \frac{\partial \exp(u_t \cdot v_k)}{\partial v_k} = \exp(u_t \cdot v_k) u_t, \quad (9)$$

这里同样用了链式法则：外层 $\exp(\cdot)$ 在 $u_t \cdot v_k$ 处求值，内层 $u_t \cdot v_k$ 对 v_k 的梯度是 u_t 。

于是

$$\frac{\partial \log Z}{\partial v_k} = f'(g) \cdot \frac{\partial g}{\partial v_k} = \frac{1}{Z} \exp(u_t \cdot v_k) u_t = P(k | t) u_t. \quad (10)$$

(iii) 合并两种情况。把 (i)(ii) 两部分相加，并引入指示函数 $\mathbb{I}[k = c]$ ：

$$\frac{\partial y}{\partial v_k} = (P(k | t) - \mathbb{I}[k = c]) u_t = \begin{cases} (P(k | t) - 1) u_t, & k = c, \\ P(k | t) u_t, & k \neq c. \end{cases} \quad (11)$$

对矩阵 V 的梯度。 $\partial y / \partial V$ 的第 k 行就是 $(\partial y / \partial v_k)^\top$ ，即

$$\left(\frac{\partial y}{\partial V} \right)_{k,:} = (\hat{y}_k - (e_c)_k) u_t^\top, \quad \text{因此} \quad \boxed{\frac{\partial y}{\partial V} = (\hat{y} - e_c) u_t^\top} \quad (12)$$

其形状为 $|V| \times d$ ，与分母布局（梯度与变量同形）一致。

1.3 题 1(c)：理解梯度 (12 分)

(i) “期望减观测”。在式 (6) 中， $-v_c$ 是**观测**：它就是本训练对中真实出现的上下文词的向量，是数据的“正确答案”； $\sum_{k \in V} P(k | t) v_k$ 是**期望**：它是模型当前预测分布 $P(\cdot | t)$ 下上下文向量的期望，即 $\mathbb{E}_{k \sim P(\cdot | t)}[v_k]$ ，求期望的对象是模型预测的上下文词分布。梯度等于“模型期望的上下文向量”减去“实际观测到的上下文向量”。

梯度为零当且仅当 $v_c = \sum_k P(k | t) v_k$ ，例如 $P(c | t) = 1$ （模型把概率全部押在正确词上）。实际训练中不可能严格达到：softmax 输出对有限参数总是严格位于 $(0, 1)$ 内， $P(c | t) < 1$ ；而且同一个中心词在不同句子中会与不同上下文词配对，这些约束互相冲突，不可能同时满足，因此梯度通常一直非零（呈噪声形式）。

(ii) 一步梯度下降的效果。固定 u_t ，只用学习率 η 更新 v_c 。由式 (11)，

$$v_c^{\text{new}} = v_c - \eta(P(c | t) - 1) u_t = v_c + \eta(1 - P(c | t)) u_t. \quad (13)$$

于是

$$u_t \cdot v_c^{\text{new}} = u_t \cdot v_c + \eta(1 - P(c | t)) \|u_t\|^2 \geq u_t \cdot v_c, \quad (14)$$

即正例对的点积（相似度）不会减小；只要 $u_t \neq 0$ 且 $P(c | t) < 1$ ，它是严格增大的。对 $k \neq c$ ， $v_k^{\text{new}} = v_k - \eta P(k | t) u_t$ ，于是

$$u_t \cdot v_k^{\text{new}} = u_t \cdot v_k - \eta P(k | t) \|u_t\|^2, \quad (15)$$

由于 softmax 下 $P(k | t) > 0$ ，竞争词与中心词的点积严格减小。也就是说，每次更新都把“中心词—真实上下文”拉近，同时把“中心词—被误认为可能的其它词”推远。经常出现在相似上下文中的词，由于接受到的更新方向相近，会在向量空间中彼此靠近，这正是“相似的词向量靠近”的来源。

(iii) 计算复杂度。 对一个训练对，前向计算需要对词表中每个 k 求点积 $u_t \cdot v_k$ 并做 softmax，共 $O(|V|d)$ 次浮点乘法；由式 (6)、(11)，梯度对 u_t 和所有 v_k 的更新同样是 $O(|V|d)$ 。因此每个训练对的总代价为 $O(|V|d)$ 。

代入 $|V| = 5 \times 10^5$ 、 $d = 300$ ：每个训练对约 $5 \times 10^5 \times 300 = 1.5 \times 10^8$ 次乘法； 10^8 个训练对共需约

$$1.5 \times 10^8 \times 10^8 = 1.5 \times 10^{16} \quad (16)$$

次浮点乘法（把梯度与前向的常数因子算进去仍是 10^{16} 量级）。

讲义第 37 页的负采样把“在 $|V|$ 个词上的多分类 softmax”换成“1 个正例 + K 个负例的二分类”：

$$y = -\log \sigma(u_t \cdot v_c) - \sum_{i=1}^K \mathbb{E}_{j \sim P(w)} \log \sigma(-u_t \cdot v_j), \quad (17)$$

每个训练对只需更新中心词与 $K + 1$ 个上下文向量，代价降为 $O((K + 1)d)$ 。以 $K = 5$ 为例，每个训练对只需 $6 \times 300 = 1800$ 次量级的乘法，相比 1.5×10^8 约加速 $|V|/(K + 1) \approx 8 \times 10^4$ 倍，使在千万级到亿级词对的语料上训练成为可能。

2 第二部分编程 (55 分)

2.1 环境与数据

机器配置：Linux 服务器，Intel(R) Xeon(R) Silver 4210R CPU @ 2.40GHz，40 核，62 GB 内存，无 GPU 需求。Python 3.10 (conda 环境 nlp2026)，依赖 numpy、scipy、matplotlib、scikit-learn、jieba、openccc-python-reimplemented。

语料：中文维基百科过滤版 pleisto/wikipedia-cn-20230720-filtered（约 25.5 万条词条，单个 JSON 约 500 MB，正文在 completion 字段）。**评测集：**wordsim-240、wordsim-297、COS960，位于 data/eval/。

训练路线：路线 A，官方 C 代码 [tmikolov/word2vec](https://github.com/tmikolov/word2vec)，commit 20c129a，由 word2vec/train.sh 自动 clone 到 output/ 并编译，源码不进入仓库。

2.2 题 2(a)：语料预处理 (10 分)

预处理脚本为 word2vec/preprocess.py，流程为：读取 JSON（数组/JSONL 均可）取 completion 字段 → OpenCC 繁体转简体 → 小写并过滤标点、空白等无意义符号 → jieba 精确模式分词 → 每行一段、词间空格分隔写入 data/corpus.txt。使用多进程(multiprocessing.Pool)按批量分词加速；分词后统计 min_count = 5 的词表，并统计 3 个评测集在词表中的覆盖率与未登录词示例，结果保存到 word2vec/results/preprocess_stats.json，统计结果见表 1 与表 2。复现命令：

```
conda activate nlp2026
python word2vec/preprocess.py
# 快速调试（只处理前 2000 条）：
python word2vec/preprocess.py --limit 2000 --output output/corpus_test.txt \
    --stats output/stats_test.json
```

表 1: 预处理统计（全量 25.5 万条词条）

项目	数值
使用的词条数	254,547
分词后总词数 (token)	82,854,433
min_count=5 的词表大小	414,976
预处理耗时 (秒)	237.5 (16 进程)

表中词表大小按预处理脚本的统计口径给出；官方 word2vec 训练日志打印的 Vocab size 为 414,977，与计数器口径相差 1 个词，不影响训练设置。

表 2: 评测集词覆盖率（未登录词比例）

数据集	唯一词数	覆盖词数	覆盖率
wordsim-240	327	326	99.69%
wordsim-297	383	377	98.43%
COS960	1,744	1,560	89.45%

未登录词示例与分析：以 min_count=5 的词表为基准，wordsim-240 仅 1 个唯一词未覆盖，wordsim-297 有 6 个，COS960 有 184 个。对典型未登录词回查全量语料 data/corpus.txt 的词频后，可以把未登录词分成四类：

- **被 jieba 切碎：**如“日本人”“美国人”“社会调查”“相似度”在语料中并不以整词出现，而是被切成“日本/人”“美国/人”“社会/调查”“相似/度”等高频片段；
- **低频词被 min_count 过滤：**如“前情”（4 次）、“做秀”（1 次）、“倔犟”（1 次）、“口涎”（2 次）、“奸笑”（4 次）、“吸顶灯”（1 次），出现次数不足 5 次；
- **语料中确实没有：**如“乖觉”“反剪”“前轮胎”“后轮胎”“心签名”等在全量语料中出现 0 次；
- **英文大小写不一致：**wordsim-297 的 OPEC 在预处理中被统一小写为 opec（语料中出现 28 次），但评测脚本按原样精确匹配，因此被误判为未登录（评测时对词对统一小写即可覆盖）。

能否用 jieba.load_userdict 提高覆盖率：可以，把评测集词表加入用户词典后，jieba 会优先把这些词切成整体，从而减少“被切碎”的 OOV。但这样做有两个问题：(i) 评测集本身被

用于分词，等于把测试信息泄漏进训练流程，评出的覆盖率与相似度会偏乐观，不再反映真实泛化；(ii) 用户词典只解决“分词粒度不一致”造成的假 OOV，如果语料中确实没有该词，仍然无法覆盖。因此正式实验中不建议用评测集构建用户词典，可在报告里作为对照实验说明。

2.3 题 2(b): 训练 (12 分)

本实验采用路线 A，训练 skip-gram + 负采样词向量 [Mikolov et al., 2013a,b]。基线超参数：维度 $d = 200$ 、窗口 $m = 5$ 、负例数 $K = 5$ 、下采样阈值 10^{-4} 、`min_count=5`、轮数 5，线程数固定为 8。训练命令（由 `train.sh` 执行）：

```
output/word2vec_official/word2vec \  
-train data/corpus.txt -output output/sg_w5.txt \  
-cbow 0 -size 200 -window 5 -negative 5 -hs 0 \  
-sample 1e-4 -min-count 5 -iter 5 -threads 8 -binary 0
```

官方 C 代码不含随机种子参数，训练过程依赖负采样表等内部随机性；本实验固定线程数与迭代轮数以保证设置可比。机器配置与完整命令如上，基线 ($m = 5$) 训练耗时 1648 秒（约 27.5 分钟，含加载与写出时间），窗口对比各设置的耗时见表 4。

训练日志保存在 `output/logs/`。官方 C 代码不输出逐轮损失，但会按训练进度记录当前学习率 (Alpha 与 Progress)，`visualize.py --which lr` 据此绘制学习率随训练进度的线性衰减曲线，见图 1。

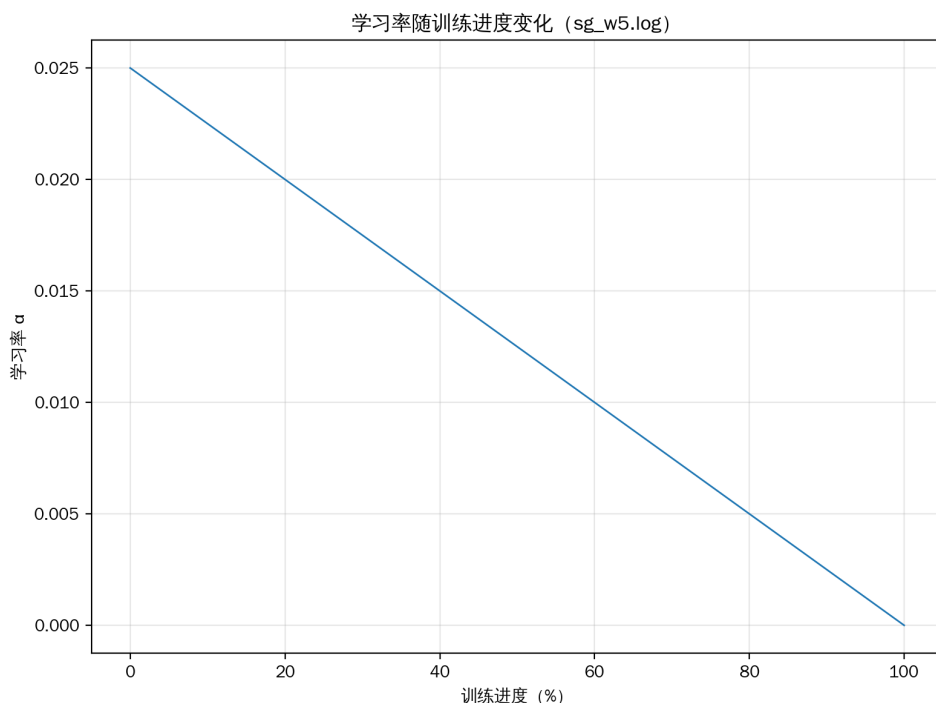


图 1: 基线训练的学习率随训练进度的线性衰减

2.4 题 2(c): 内在评测 (15 分)

评测脚本 `word2vec/evaluate.py` 自行实现了余弦相似度（对词向量矩阵做 ℓ_2 归一化后矩阵乘）、最近邻检索（一次矩阵-向量乘法，排除查询词自身）与 Spearman 相关（调用 `scipy.stats.spearmanr`），未使用 `gensim` 的评测函数。三个评测集分别来自 [Jin and Wu \[2012\]](#)（`wordsim-240/297`）与 [Huang et al. \[2019\]](#)（`COS960`）。运行：

```
python word2vec/evaluate.py --vectors output/sg_w5.txt
```

(i) 词相似度。 结果见表 3。

表 3: 基线词向量的词相似度结果（OOV 词对跳过）

数据集	总对数	覆盖对数	Spearman ρ
wordsim-240	240	239	0.5457
wordsim-297	297	292	0.5961
COS960	960	801	0.4453

三个数据集上均取得中等偏上的正相关（ $\rho \approx 0.45\text{--}0.60$ ）。`wordsim` 系列衡量“相关性”（如“李白—诗”），而 `COS960` 只衡量“近义程度”，且包含大量仅相关而非近义的词对，因此 ρ 略低是符合预期的。

(ii) 最近邻。 固定查询词：北京、语言、苹果、计算机、足球、孔子、春节、快乐；自选查询词：小米、银行、同志、打（多义词）。各查询词余弦最近的 10 个词如下（括号内为余弦值，完整结果见 `results/neighbors_sg_w5.tsv`）：

- **北京**：北京市 0.698、上海 0.678、沈阳 0.668、北京城 0.652、天津 0.633、燕翔 0.629、呼和浩特 0.628、工人体育场 0.624、昆仑饭店 0.620、海淀区 0.616
- **语言**：语法 0.768、书面语言 0.759、语等 0.752、第二语言 0.741、母语 0.740、语是 0.739、语系 0.738、语法结构 0.738、语族 0.737、语 0.734
- **苹果**：黑莓 0.649、坚果 0.639、小米 0.632、火龙果 0.624、脆片 0.621、南瓜 0.618、主打产品 0.610、草莓 0.604、洋葱 0.603、鳄梨 0.598
- **计算机**：图形学 0.799、计算机网络 0.787、人机交互 0.785、计算机系统 0.785、信号处理 0.773、应用软件 0.772、计算机辅助 0.769、分布式计算 0.761、电脑 0.759、电子计算机 0.758
- **足球**：甲级联赛 0.777、俱乐部 0.735、篮球 0.733、足球联赛 0.732、国际足球 0.731、联赛 0.725、球队 0.717、司职 0.714、五人制 0.709、青年队 0.705
- **孔子**：颜回 0.654、孟子 0.626、曾子 0.615、孔丘 0.613、子思 0.613、十哲 0.603、`confuciusornis` 0.601、圣贤 0.599、颜何 0.598、家语 0.595

- **春节**：联欢晚会 0.789、元宵 0.738、央视春晚 0.718、大年初一 0.683、连休 0.677、春节假期 0.673、除夕夜 0.672、郭冬临 0.671、中秋节 0.665、春节晚会 0.660
- **快乐**：锋尚 0.663、男声 0.659、幸福 0.648、爸爸妈妈 0.647、敢不敢 0.643、强合辑 0.640、好时光 0.637、谁动 0.634、相亲相爱 0.634、一生一世 0.626
- **小米**：红米 0.719、miui 0.692、pro2s 0.646、smartisan 0.642、苹果 0.632、新品 0.625、oneplus 0.623、手机 0.607、雷军 0.606、旗舰机 0.606
- **银行**：商业银行 0.815、分行 0.813、总行 0.766、金融机构 0.747、兴业银行 0.743、中国银行 0.739、储蓄银行 0.734、bank 0.733、该行 0.732、国家银行 0.726
- **同志**：党外 0.581、祖国统一 0.574、老战士 0.562、平权 0.559、恳谈会 0.552、积极分子 0.550、少先队 0.547、女同性恋 0.546、台湾同胞 0.542、保卫和平 0.542
- **打**：席面 0.651、平飞安 0.650、打打 0.643、一个打 0.633、廖健富 0.632、安打及 0.632、点圈 0.626、打以 0.626、lofton 0.624、球击 0.623

“意外”近邻与解释：

(i) **苹果**的近邻以水果为主（黑莓、火龙果、草莓、鳄梨），却混入了“小米”；而查询**小米**的近邻几乎全是手机品牌（红米、miui、雷军、苹果、oneplus）。“苹果”“小米”“黑莓”都是“水果/科技品牌”双义词：词向量把两个义项的语境平均在同一个向量里，而维基语料中“小米公司”的语料远多于“小米（谷物）”，因此小米向量偏向科技义项，苹果向量则偏水果义项，但两者仍因共同义项互相靠近。

(ii) **孔子**的近邻中出现拉丁学名 *confuciusornis*（孔子鸟），它在语料中仅出现 9 次，几乎只与“孔子”共现（该属名用以纪念孔子），这是“共现联想”而非语义相似造成的近邻。

(iii) **春节**的近邻包含“郭冬临”（仅出现 18 次的演员名），原因是他是央视春晚的常客，二者上下文高度重叠；**快乐**的近邻（锋尚、强合辑、谁动、相亲相爱等）多来自“快乐男声”“快乐大本营”等节目与歌曲名，属于固定搭配造成的伪近邻。

(iv) **打**作为高频多义动词，近邻大量是分词碎片（“安打及”“打以”“一个打”）和棒球术语（安打、球击、lofton、廖健富），说明高频虚化动词的向量被多种语境稀释，单独一个向量难以承载其全部义项。相比之下，语义较单一的“银行”“计算机”近邻质量很高（商业银行、分行、总行；图形学、人机交互、电脑）。

(iii) **误差分析**。按模型余弦排名与人工打分排名的名次差（rankdata 平均秩）排序，COS960 上差异最大的 5 对（保存于 results/cos960_errors_sg_w5.tsv）及原因分析：

- **聪明绝顶—胆小怕事**（人工 0.00，余弦 0.844，模型第 47 名 / 人工第 746 名）：两者都是描写人物性格的四字成语，常出现在“他是一个……的人”等相同句式，语料中仅 10 与 16 次。模型只看到语境相似，而 COS960 标注的是“近义”，二者只是“相关”，属于数据集语义与共现语义的错位。
- **困倦—烦躁**（人工 0.00，余弦 0.843）：都是身体/情绪状态词，共享“感到”“有些”语境；另外“烦躁”是“烦躁”的异形写法（语料中仅 7 次，而“烦躁”有 73 次），低频异形词的向量估计本就不稳。

- **球队—赛季** (人工 0.00, 余弦 0.807): 体育领域高相关词 (语料中分别出现 36 085 与 20 261 次), 上下文高度重叠, 但“相关”不等于“近义”, 再次体现 COS960 与 wordsim 的差别。
- **悲鸣—哀鸣** (人工 3.87, 余弦 0.462, 模型第 678 名 / 人工第 41 名): 真正近义的书面语词, 但“哀鸣”在语料中仅出现 17 次 (“悲鸣” 128 次), 低频词的向量方向不稳定, 且两词常在文体的不同上下文中单独出现。
- **父母—爸妈** (人工 4.00, 余弦 0.514, 模型第 617 名 / 人工第 11 名): 语义近义但语体不同, “爸妈”口语化且仅出现 65 次 (“父母” 6931 次), 二者上下文分布有明显差异 (“父母”多用于书面与正式叙述)。

总体来看, 模型错误的两个主要来源是: (1) 训练目标捕获的是“共现/相关”而非“近义”, 因此“球队—赛季”“聪明绝顶—胆小怕事”这类相关词对被判为高度相似; (2) 低频词与异形词 (哀鸣、爸妈、烦躁) 的向量估计不稳定。

2.5 题 2(d): 超参数对比 (10 分)

选择窗口大小对比: $m \in \{2, 5, 10\}$ ($m = 5$ 为基线), 其余超参数保持不变, 即 `bash word2vec/train.sh window`。各设置在 3 个评测集上的结果与训练耗时见表 4 与图 2:

表 4: 不同窗口大小的结果与训练耗时 (秒)

设置	ρ (240)	ρ (297)	ρ (COS960)	训练耗时
$m = 2$	0.5134	0.5668	0.4587	≈ 900 (估算)
$m = 5$ (基线)	0.5457	0.5961	0.4453	1648
$m = 10$	0.5484	0.5855	0.4158	2480

其中 $m = 2$ 的耗时未由 `train.sh` 精确记录 (当时脚本尚未加入计时), 按训练日志的吞吐量 (约 6.2×10^4 词/线程/秒) 估算; 其余为脚本记录值。

分析: 窗口增大时, “相关性”与“近义性”的变化方向并不一致: wordsim-240 随窗口增大单调上升 ($0.5134 \rightarrow 0.5457 \rightarrow 0.5484$), wordsim-297 在 $m = 5$ 达到峰值 ($0.5668 \rightarrow 0.5961 \rightarrow 0.5855$), 而 COS960 随窗口增大单调下降 ($0.4587 \rightarrow 0.4453 \rightarrow 0.4158$)。这与 Levy et al. [2015] 的结论一致: 大窗口使向量编码更多“话题/领域相关”, 有利于 wordsim 这类相关性评测; 小窗口聚焦紧邻语境, 更能捕获可相互替换的“同义/同功能”关系, 因而更有利于 COS960 的近义评测。训练耗时随窗口近似线性增长, $m = 10$ 为 2480 秒, 约为 $m = 5$ (1648 秒) 的 1.5 倍。

不同设置下近邻词的变化以“足球”最为典型: $m = 2$ 时前 5 个近邻为篮球 0.74、排球 0.70、足球运动 0.67、五人制 0.67、足球队 0.66, 以“同类可替换的球类运动”为主; $m = 10$ 时为甲级联赛 0.83、足球联赛 0.80、俱乐部 0.79、联赛 0.77、球队 0.77, 全部是同一话题领域的联赛/俱乐部词, 余弦也更高。这说明窗口越大, 模型越偏“话题相关”, 越小越偏“功能/类别相似”。“苹果”也类似: $m = 2$ 的近邻是橘子、马铃薯、柑橘、椰树 (同类食物), $m = 10$ 时为黑莓、水果、南瓜、软果 (仍偏食物, 但更偏共现话题)。

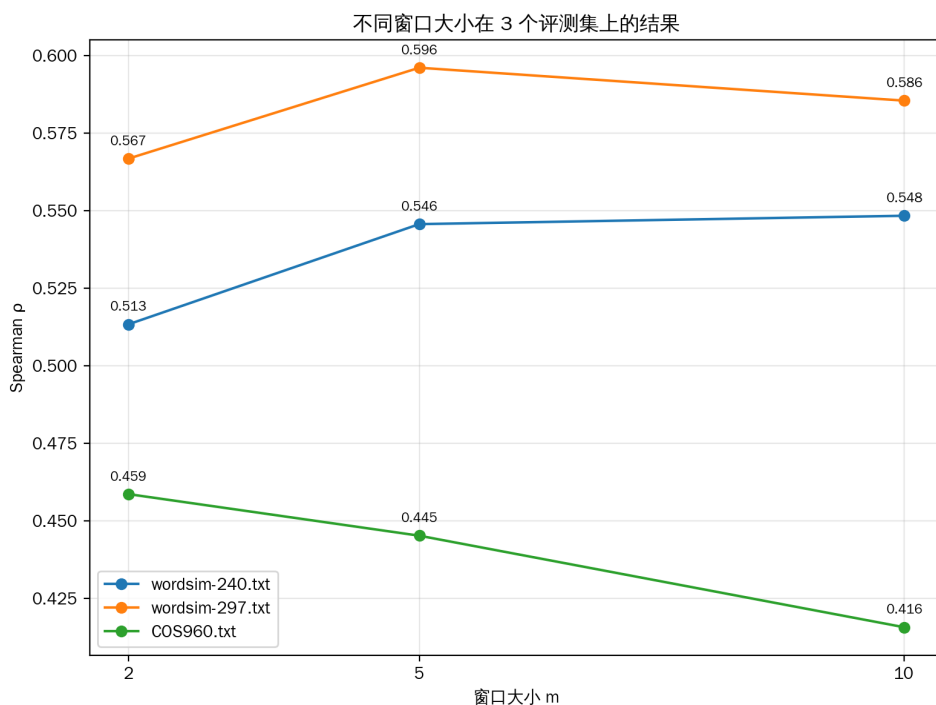


图 2: 窗口大小对 3 个评测集 Spearman ρ 的影响

2.6 题 2(e): 可视化 (8 分)

可视化脚本 `word2vec/visualize.py` 选取 7 个语义类别 (地区、动物、颜色、朝代、职业、体育、学科) 共 81 个词, 分别用 PCA 与 t-SNE 降到 2 维并按类别着色, 结果见图 3 与图 4。这 81 个词在 `data/corpus.txt` 中的词频均不低于 5, 实际全部保留在词表中 (81/81), 满足 ≥ 5 类、 ≥ 50 词的要求。

讨论:

(i) **PCA 与 t-SNE 的类别分离度**: 按类别计算二维坐标的轮廓系数, PCA 为 0.425, t-SNE 为 0.644 (`perplexity = 27`、`seed = 42`), t-SNE 明显分得更开。图 3 中颜色、地区等类别已经分开, 但体育、职业、学科三类在下方互相交叠; 图 4 中 7 个类别几乎各自成簇, 只有职业 (教师、记者、律师等) 与学科略有交叠, 符合 “教师/记者/律师” 与学科语境相关的直觉。

(ii) **t-SNE 簇间距离不可直接解读为语义距离**: t-SNE 只保留局部邻域结构, 簇间相对位置与距离随参数变化很大。对比图 4 (`perplexity = 27`) 与图 6 (`perplexity = 10`): 两次运行中 “地区” 簇分别位于画布左侧与右侧, “颜色” 簇分别在上方与右下, 但各簇内部始终紧密; 因此只能解读 “簇内/近邻关系”, 不能把两个簇的间距当作语义相似度。

(iii) **稳定性**: 改变随机种子 (`seed 42` $\rightarrow$ 0, 图 7) 后, 全局布局再次整体旋转/平移, 轮廓系数从 0.644 变为 0.631; 改变 `perplexity` 后为 0.684。类别内部始终聚在一起, 但个别词的位置 (如 “北京” 在地区簇与朝代簇之间的位置) 会在不同运行间移动, 说明 t-SNE 用于观察局部结构是可靠的, 但不宜过度解读具体坐标。

(iv) **近邻群是否重叠**: 图 5 中, 北京、足球、春节、计算机、孔子、苹果 6 组基本互不重叠, 各自内部紧凑: 北京组包含 “天津、上海、工人体育场、昆仑饭店” 等城市与地标; 孔子组 (颜回、孟子、曾子) 独立于其他组。唯一的 “跨界” 是苹果组混入 “小米”, 与题 2(c)(ii) 中观察到

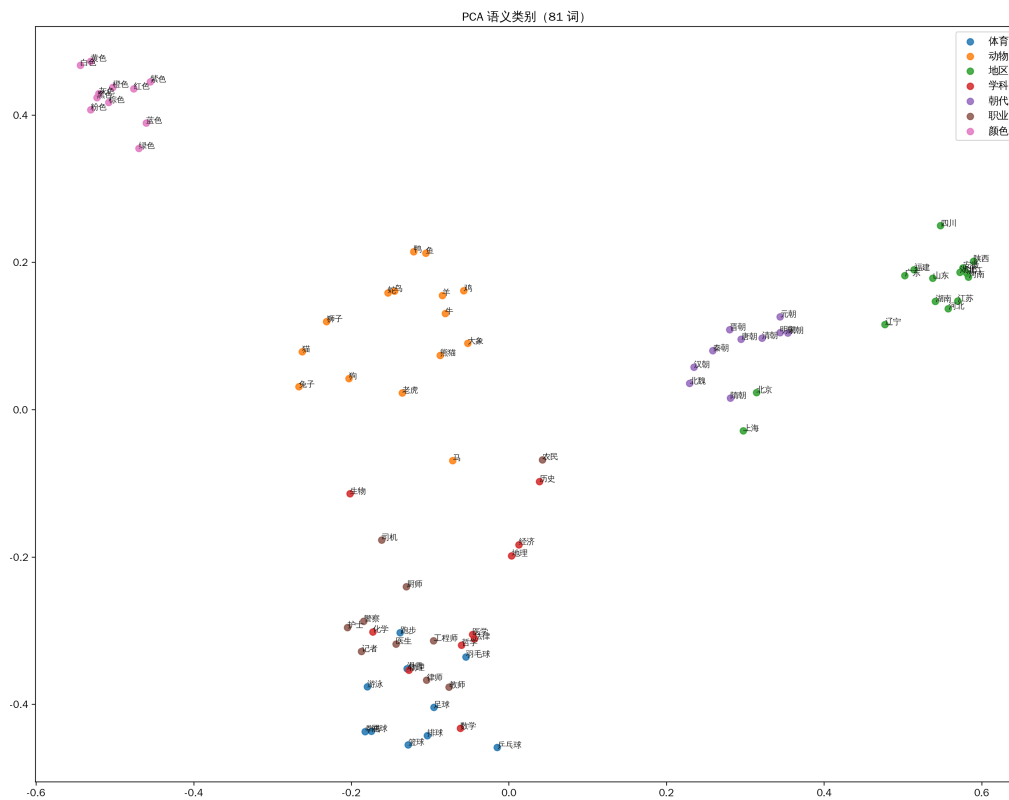


图 3: 语义类别的 PCA 二维散点图

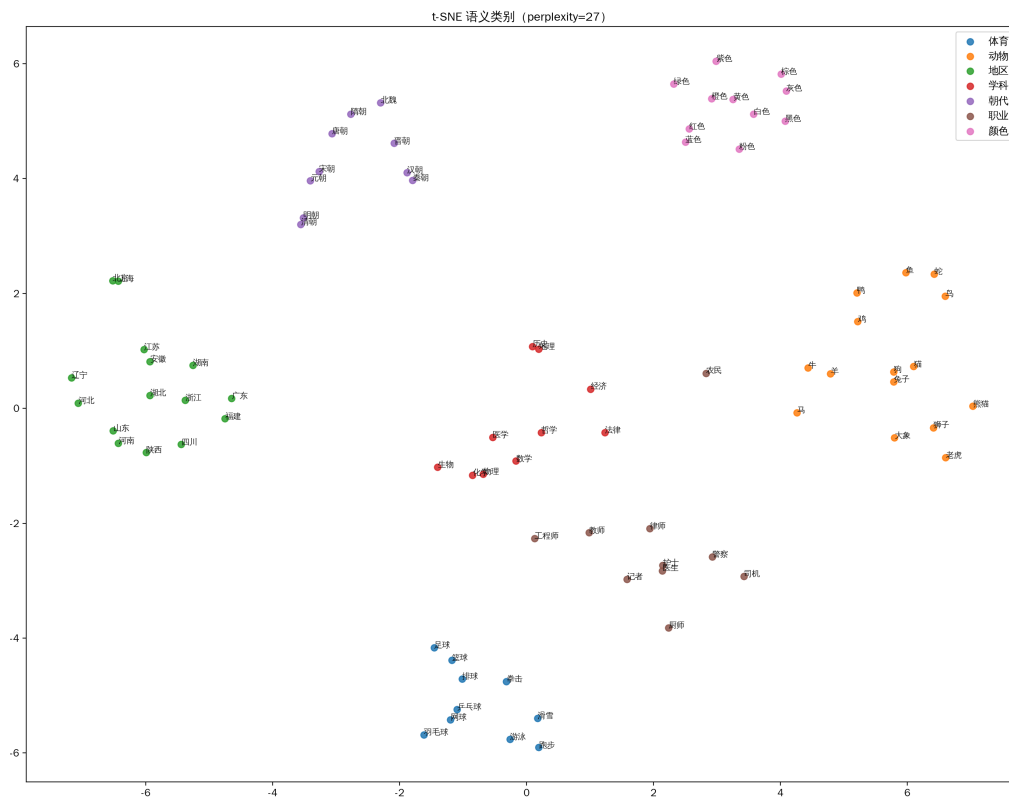


图 4: 语义类别的 t-SNE 二维散点图

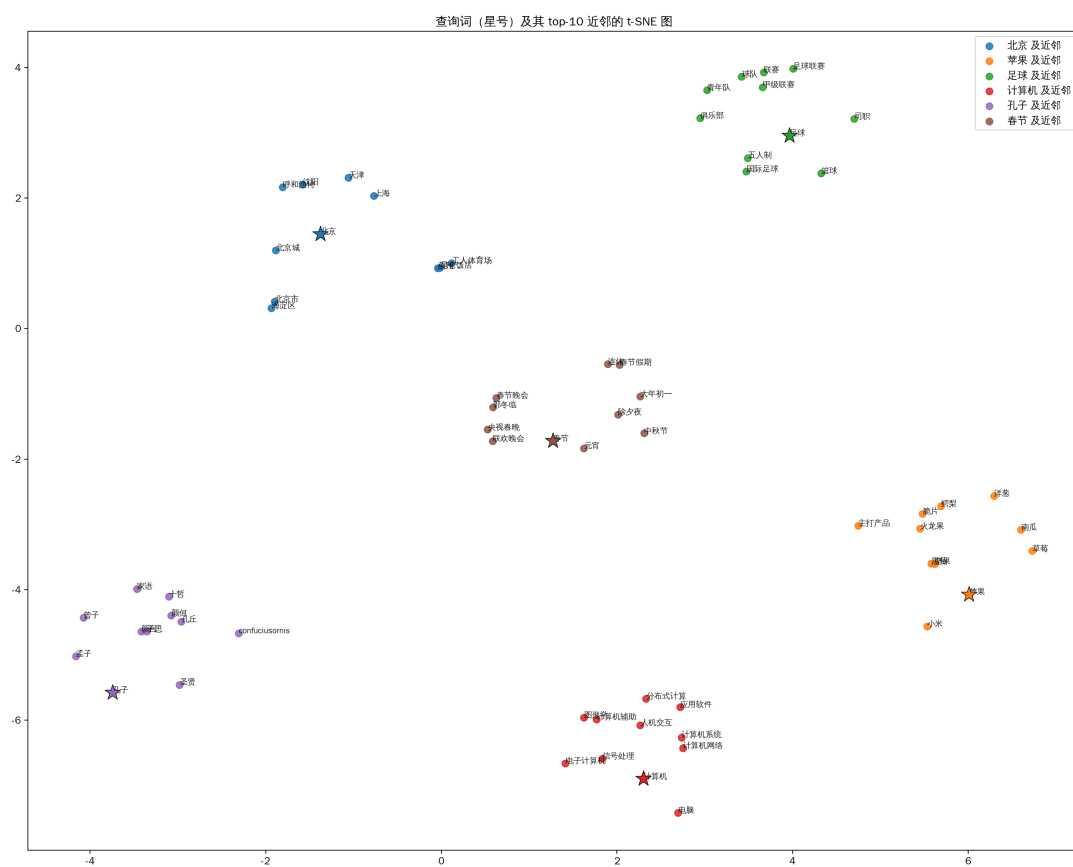


图 5: 查询词（星号）及其 top-10 近邻的 t-SNE 图

的“水果/科技品牌”双义现象一致。

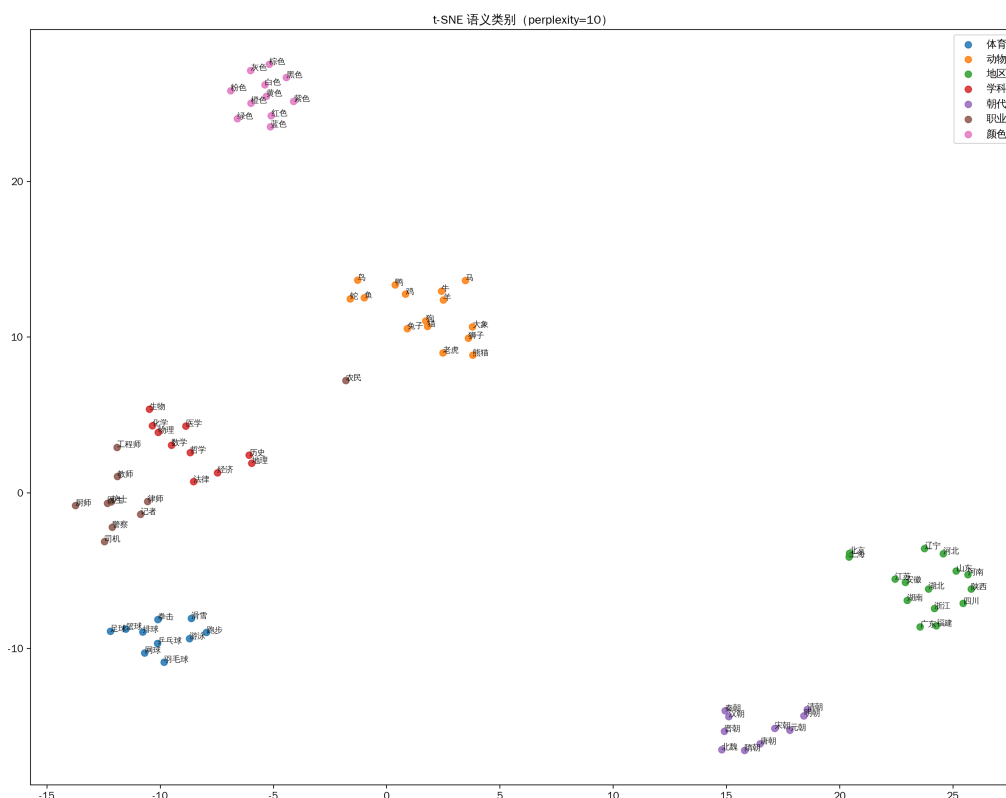


图 6: perplexity = 10 时的 t-SNE 图（与图 4 对比簇间位置变化）

3 第三部分提交信息

- 仓库地址: <https://github.com/acerdream/nlp-fall2026-lab4>
- 最后一次提交的 commit id: 95f4350
- 训练路线: A (tmikolov/word2vec @ 20c129a)

参考文献

Junjie Huang, Fanchao Qi, Chenghao Yang, Zhiyuan Liu, and Maosong Sun. COS960: A chinese word similarity dataset of 960 word pairs, 2019.

Peng Jin and Yunfang Wu. SemEval-2012 task 4: Evaluating chinese word similarity. In *Proceedings of the 6th International Workshop on Semantic Evaluation (SemEval)*, pages 374–377, 2012.

Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Speech Recognition, and Computational Linguistics*. Draft edition online, 3rd edition, 2024. URL <https://web.stanford.edu/~jurafsky/slp3/>.

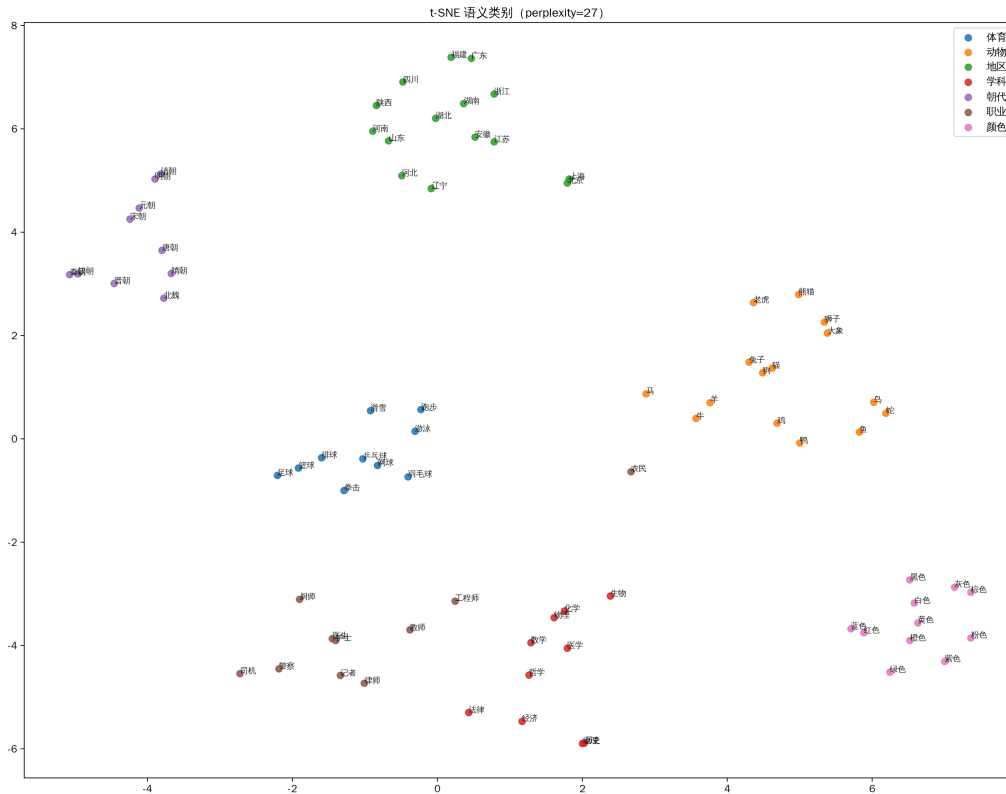


图 7: 随机种子 = 0 时的 t-SNE 图 (与图 4 对比稳定性)

Omer Levy, Yoav Goldberg, and Ido Dagan. Improving distributional similarity with lessons learned from word embeddings. *Transactions of the Association for Computational Linguistics (TACL)*, 3:211–225, 2015.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *Proceedings of the International Conference on Learning Representations (ICLR) Workshop*, 2013a.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 3111–3119, 2013b.